

連結リスト

図解：連結リスト入門

連結リストとは？

ノードが鎖のようにつながる動的データ構造
ポインタで次の要素を指し、実行中にサイズを柔軟に変更できます。

連結リストの構造

① ノード (Node) : リストの最小単位

② ヘッドポインタ (Head Pointer) : リストへの入り口
常にリストの最初のノードを指し示す、極めて重要なポインタです。

全体像のイメージ
ヘッドポインタを起点に、各ノードが次々と連結されています。

連結リストと配列の比較

	連結リスト	配列
① サイズ	動的に変更可能	固定サイズ
② 要素の追加/削除	効率的 (特に途中)	非効率 (後続要素のシフト要)
③ 要素へのアクセス	先頭から順に辿る	インデックスで直接アクセス

イントロダクション

コンピュータサイエンスの世界において、連結リストはポインタを駆使する最も基本的かつ重要な動的データ構造の一つです。データを効率的に管理する方法を学ぶことは、優れたソフトウェアエンジニアになるための第一歩です。この資料を通じて、連結リストの概念を深く理解し、C言語での実装スキルを習得することで、皆さんが将来より複雑な課題に取り組むための強固な基盤を築くことを目的としています。

1. 連結リストとは何か？

概念定義

連結リストとは、動的に確保されたノード (node) と呼ばれる要素の集合体です。各ノードは「データ」と「次のノードを指すポインタ」を保持しており、このポインタを介してノード同士が鎖のように連結されることで、一つのリスト構造を形成します。この構造により、プログラムの実行中にリストのサイズを柔軟に変更することが可能です。

配列との比較分析

連結リストは、しばしば配列と比較されます。どちらもデータを連続的に格納する構造ですが、その内部的な仕組みと特性には大きな違いがあります。

比較項目	連結リスト	配列
サイズ	動的にサイズを変更可能。初期サイズを定義する必要がない。	固定サイズ。最初にサイズを定義する必要がある。

比較項目	連結リスト	配列
要素の追加/削除	リストの途中での追加・削除が効率的。	途中の要素を追加・削除するには、後続の要素を全てシフトさせる必要があり、コストが高い。
要素へのアクセス	先頭から順に辿る必要がある（シーケンシャルアクセス）。n番目の要素へのアクセスは非効率。	インデックスを指定して直接アクセスできる（ランダムアクセス）。
メモリオバーヘッド	各ノードがデータに加えて次へのポインタを保持するため、オーバーヘッドが大きい。	データのみを格納するため、メモリ効率が良い。
実装の複雑さ	ポインタと動的メモリ確保が必要なため、実装が複雑になり、メモリリーク等のリスクがある。	実装が比較的容易。

前提知識

この章の説明を完全に理解するためには、以下のC言語の概念に精通している必要があります。もし不安な点があれば、先に復習しておくことを強く推奨します。

- ポインタ: メモリアドレスを扱うための基本的な概念と操作。
- 動的メモリ確保: malloc や free を用いた、プログラム実行中のメモリ確保・解放。
- 構造体: 関連する複数のデータを一つのまとまりとして扱うための仕組み。

連結リストの基本的な概念と、配列との違いを理解したところで、次のセクションではその具体的な構造についてさらに詳しく見ていきましょう。

2. 連結リストの構造

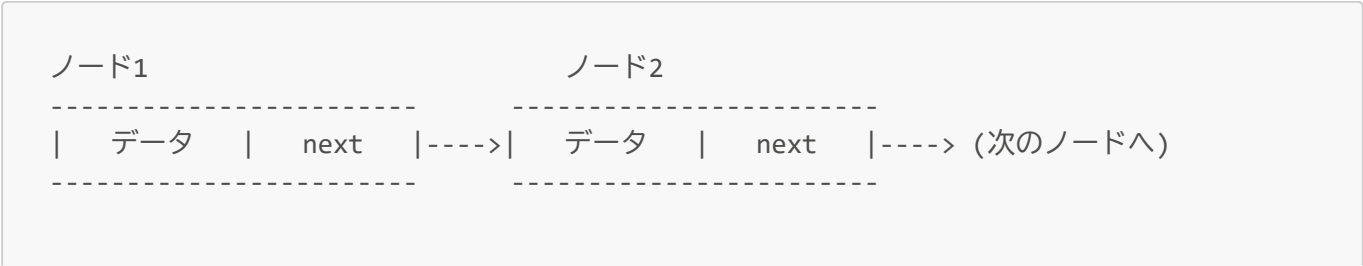
このセクションでは、連結リストを構成する基本的な要素である「ノード」と、リスト全体を管理する「ヘッドポインタ」について、C言語での実装を交えながら理解を深めていきます。

ノード (Node)

連結リストの最小構成単位がノードです。各ノードは、大きく分けて2つの部分から構成されます。

- データ (val): ノードが保持する値そのものです。整数、文字、あるいはより複雑な構造体など、様々な型のデータを格納できます。
- 次へのポインタ (next): リスト内の次のノードを指し示すポインタです。このポインタによってノード同士が連結されます。リストの最後のノードの場合、このポインタは NULL を指します。

テキストで図示すると、以下のようなイメージになります。



nextはノード2のポインタ
(ノード2の先頭アドレス)

nextは次のノードのポインタ
(次のノードの先頭アドレス)

C言語でのノード定義

C言語では、構造体を用いてノードを定義します。特筆すべきは、構造体自身の型へのポインタをメンバとして持つ「自己参照構造体」として定義できる点です。

```
typedef struct node {
    int val;
    struct node* next;
} node_t;
```

この定義により、node_t という型で連結リストのノードを扱うことができるようになります。

ヘッドポインタ (Head Pointer)

ヘッドポインタ (head) は、連結リストの最初のノードを指す特別なポインタです。このポインタがリスト全体への唯一のエントリーポイントとなるため、極めて重要です。リストを辿ったり、新しいノードを追加したりする操作は、すべてこの head ポインタから始まります。

テキストで図示すると、以下のようなイメージになります。

ヘッドポインタ	ノード1
-----	-----
head ----->	データ next -----> (ノード2へ)
-----	-----
headはノード1のポインタ (ノード1の先頭アドレス)	nextはノード2のポインタ (ノード2の先頭アドレス)

もし head ポインタが NULL を指している場合、それはリストにノードが一つも存在しない、つまり「空のリスト」であることを意味します。

最初のノードの作成

実際にコードを見て、最初のノードを作成するプロセスを確認しましょう。

疑似言語

```
node_t: head ← NULL;

head.val ← 1;
head.next ← NULL;
```

```

node_t * head = NULL;
head = (node_t *) malloc(sizeof(node_t));

// mallocが成功したか必ずチェックする
if (head == NULL) {
    // メモリ確保失敗時のエラー処理
    return 1;
}

head->val = 1;
head->next = NULL;

```

各行が何をしているのか、順を追って解説します。

1. `node_t * head = NULL;`

- `head` ポインタを宣言し、リストが空であることを示す `NULL` で初期化します。

2. `head = (node_t *) malloc(sizeof(node_t));`

- `malloc` 関数を呼び出し、`node_t` 構造体1つ分のメモリ領域を動的に確保します。確保されたメモリの先頭アドレスが `head` ポインタに代入されます。

3. `if (head == NULL)`

- `malloc` はメモリ確保に失敗すると `NULL` を返すため、必ずこのチェックを行います。これを怠ると、確保されていないメモリにアクセスしようとしてプログラムがクラッシュする原因となります。

4. `head->val = 1;`

- 確保したメモリ領域にアクセスし、`val` メンバに値 `1` を設定します。`head` は `node1` へのポインタなので、`head->val` は `node1` のデータを指します。

5. `head->next = NULL;`

- このノードがリストの唯一のノード（つまり末尾のノード）であるため、`next` ポインタに `NULL` を設定します。

これで、連結リストの静的な構造と最初のノードを作成する方法を理解できました。次章では、このリストを動的に操作するための具体的なアルゴリズムとコードについて学んでいきます。

3. 連結リストの基本操作

このセクションでは、連結リストを実用的に利用するために不可欠な、データの探索、追加、削除、そしてメモリ管理といった基本的な操作方法を、具体的なC言語のコード例と共に学んでいきます。堅牢なコードを書くためには、様々な「エッジケース」（例：空のリストに対する操作）を正しく処理することが重要です。

3.1 リストの探索（イテレーション）

リストに格納された全てのデータを順に処理するためには、リストを先頭から末尾まで辿る（イテレーションする）必要があります。

アルゴリズム

1. 作業用のポインタ変数（currentなど）を用意し、head ポインタで初期化します。
2. current ポインタが NULL でない間、以下の処理を繰り返します。 a. current が指すノードに対して処理（値の表示など）を行います。 b. current ポインタを、current->next が指す次のノードへ進めます。
3. current ポインタが NULL になったら、リストの末尾に到達したことを意味し、ループを終了します。

疑似言語

```
node_t: current ← head;

while (currentが未定義でない)
    current.valを表示
    current ← current.next
endwhile
```

コード例

以下は、リストの全てのノードの値を表示する print_list 関数です。

```
void print_list(node_t * head) {
    node_t * current = head;

    while (current != NULL) {
        printf("%d -> ", current->val);
        current = current->next;
    }
    printf("NULL\n");
}
```

この関数は、current ポインタを使ってリストの先頭から一つずつノードを辿り、val を出力し、next ポインタで次のノードへ移動する、という処理を current が NULL になるまで繰り返します。

エッジケースの考察: もしこの関数に NULL の head ポインタ（空のリスト）が渡された場合、while ループの条件 (current != NULL) が最初から偽となり、ループは一度も実行されません。結果として安全に "NULL" とだけ表示して終了するため、このコードは空のリストを正しく扱っています。

3.2 要素の追加（挿入）

3.2.1 リストの末尾への追加

リストの最後に新しい要素を追加する操作です。

アルゴリズム

1. 新しいノードのためのメモリを malloc で確保し、値と next ポインタ (NULL) を設定します。
2. エッジケース: リストが空（head が NULL）の場合、新しいノードをリストの先頭にします。

3. リストが空でない場合、作業用の current ポインタでリストの最後のノードまで移動します。
4. 最後のノードの next ポインタが、新しいノードを指すように設定します。

疑似言語

```

○append_node(node_t: head, 整数型 val)
    node_t: new_node

    new_node.val ← val;
    new_node.next ← NULL;

    // リストが空の場合、新しいノードをheadにして終了
    if (headが未定義)
        head ← new_node;
        return;
    endif

    // リストが空でない場合
    node_t: current ← head
    // 最後のノードまで移動
    while (current.nextが未定義でない)
        current ← current.next;
    endwhile

    // 最後のノードの次に新しいノードを連結
    current.next ← new_node;

```

コード例

この操作は、リストが空の場合に head ポインタ自体を変更する必要があるため、要素を追加する関数には、headポインタを **参照渡し** しなければなりません。そのため引数にはダブルポインタ (node_t **head) を取ります。

```

void append_node(node_t **head, int val) {
    node_t *new_node = malloc(sizeof(node_t));
    if (new_node == NULL) {
        // メモリ確保失敗
        return;
    }
    new_node->val = val;
    new_node->next = NULL;

    // リストが空の場合、新しいノードをheadにする
    if (*head == NULL) {
        *head = new_node;
        return;
    }

    node_t *current = *head;
    // 最後のノードまで移動

```



```

while (current->next != NULL) {
    current = current->next;
}

// 最後のノードの次に新しいノードを連結
current->next = new_node;
}

```

【重要】ポインタへのポインタ (ダブルポインタ) の役割

append_node 関数の引数が node_t ** head となっている点に、あらためて注目してみましょう。これは「node_t 型へのポインタ」へのポインタ、つまりダブルポインタです。

なぜダブルポインタが必要なのでしょう？ 第2部の第3章『引数の参照渡し』で学んだことを思い出してください。C言語では、関数に渡された引数はコピー（値渡し）されます。もし引数が node_t * head であった場合、関数内で head の値を変更しても、その変更は関数内でのみ有効となり、呼び出し元の head ポインタは変わりません。しかし、push 操作ではリストの先頭が新しいノードに変わるため、呼び出し元の head ポインタそのものを書き換える必要があります。これを実現するために、head ポインタのアドレスを関数に渡し、関数内から間接的に呼び出し元の head ポインタの値を変更するのです。

3.2.2 リストの先頭への追加 (Push)

リストの先頭に新しい要素を追加する操作です。スタック構造における push 操作に相当します。

アルゴリズム

1. 新しいノードを作成し、値を設定します。
2. 新しいノードの next ポインタが、現在のリストの先頭 (head) を指すようにします。
3. head ポインタ自体を、新しく作成したノードを指すように更新します。

疑似言語

```

Opush(node_t: head, 整数型: val)
    node_t: new_node

    new_node.val ← val;
    new_node->next ← head;
    head ← new_node;

```

コード例

```

void push(node_t ** head, int val) {
    node_t * new_node;
    new_node = (node_t *) malloc(sizeof(node_t));
    // mallocの失敗をチェック
    if (new_node == NULL) {
        return;
    }
}

```

```

    new_node->val = val;
    new_node->next = *head;
    *head = new_node;
}

```

3.3 要素の削除

3.3.1 リストの先頭から削除 (Pop)

リストの先頭要素を削除する操作です。スタック構造における pop 操作に相当します。

アルゴリズム

1. エッジケース: リストが空の場合は何もせず、エラーを示す値を返します。
2. head が指す次のノード（2番目のノード）のアドレスを一時的なポインタに保存します。
3. 現在の head が指す先頭ノードのメモリを free() で解放します。
4. head ポインタを、ステップ2で保存しておいた2番目のノードを指すように更新します。

疑似言語

```

整数型 pop(node_t: head)
  整数型: retval ← -1;
  node_t: next_node ← 未定義の値

  // エッジケース: 空のリスト
  if (headが未定義であれば)
    return -1; // 異常終了
  endif

  next_node ← head.next
  retval ← head.val
  head ← next_node

  return retval;

```

コード例

```

int pop(node_t ** head) {
    int retval = -1;
    node_t * next_node = NULL;

    // エッジケース: 空のリスト
    if (*head == NULL) {
        return -1;
    }

    next_node = (*head)->next;
    retval = (*head)->val;

```



```

    free(*head);
    *head = next_node;

    return retval;
}

```

このコードは、`*head == NULL` をチェックすることで、空のリストに対して安全に動作します。これは防御的プログラミングの良い実践例です。

3.3.2 リストの末尾から削除

アルゴリズム

リストの末尾を削除するには、末尾から2番目のノードを見つける必要があります。なぜなら、最後のノードを削除した後、その一つ手前のノードの `next` ポインタを `NULL` に更新する必要があるからです。

疑似言語

```

○整数型: remove_last(node_t: head)
  整数型 retval ← -1
  node_t: current

  // エッジケース: リストが空
  if (headが未定義)
    return -1;
  endif

  // エッジケース: リストに要素が1つだけ
  if (head.nextが未定義)
    retval ← head.val
    head ← 未定義の値 // 呼び出し元のheadを未定義に更新
    return retval;
  endif

  // 末尾から2番目のノードまで移動
  current ← head
  while ((current.next).nextが未定義でない)
    current ← current.next
  endwhile

  // current は末尾から2番目のノードを指している
  retval = (current.next).val
  current.next ← 未定義の値

  return retval;

```

コード例

この操作も、リストに要素が1つしかない場合に `head` ポインタ自体を `NULL` に変更する必要があるため、ダブルポインタを引数に取ります。

```

int remove_last(node_t ** head) {
    int retval = -1;

    // エッジケース: リストが空
    if (*head == NULL) {
        return -1;
    }

    // エッジケース: リストに要素が1つだけ
    if ((*head)->next == NULL) {
        retval = (*head)->val;
        free(*head);
        *head = NULL; // 呼び出し元のheadをNULLに更新
        return retval;
    }

    // 末尾から2番目のノードまで移動
    node_t * current = *head;
    while (current->next->next != NULL) {
        current = current->next;
    }

    // current は末尾から2番目のノードを指している
    retval = current->next->val;
    free(current->next);
    current->next = NULL;

    return retval;
}

```

以前の実装では、要素が1つのリストを削除すると、呼び出し元の head は解放済みのメモリを指し示す「ダングリングポインタ」となり、非常に危険でした。ダブルポインタを使うことで、*head = NULL; と安全にリストを空にすることができます。

3.3.3 指定したインデックスの要素を削除

アルゴリズム

1. 削除したいノードの一つ手前のノードまでイテレーションで移動します。
2. 削除対象のノード（手前のノードの next が指すノード）を一時的なポインタに保存します。
3. 手前のノードの next ポインタを、削除対象ノードの next ポインタが指していたノードに繋ぎ変えます。これにより、リストから削除対象ノードがバイパスされます。
4. ステップ2で保存した一時ポインタを使い、削除対象ノードのメモリを free() で解放します。

疑似言語

```

○整数型: remove_by_index(node_t: head, 整数型: n)
    node_t: current ← head
    node_t: temp_node ← NULL

```

```

整数型: i ← 0
整数型: retval ← -1;

// エッジケース: 先頭(インデックス0)を削除する場合
if (nが0)
    return pop(head);
endif

// 削除対象の「一つ手前」のノードまで移動
for (i = 0; i < n - 1; i++) {
    if (current->next == NULL) {
        return -1; // インデックスが範囲外
    }
    current = current->next;
}

// 削除対象のノードが存在しない場合 (n-1番目はあるがn番目はない)
if (current.nextが未定義)
    return -1;

temp_node ← current.next
retval ← temp_node.val
current.next ← temp_node.next // リンクの繋ぎ換え

return retval

```

コード例

```

int remove_by_index(node_t ** head, int n) {
    // エッジケース: 先頭(インデックス0)を削除する場合
    if (n == 0) {
        return pop(head);
    }

    node_t * current = *head;
    node_t * temp_node = NULL;
    int i = 0;
    int retval = -1;

    // 削除対象の「一つ手前」のノードまで移動
    for (i = 0; i < n - 1; i++) {
        if (current->next == NULL) {
            return -1; // インデックスが範囲外
        }
        current = current->next;
    }

    // 削除対象のノードが存在しない場合 (n-1番目はあるがn番目はない)
    if (current->next == NULL) {
        return -1;
    }
}

```

```
temp_node = current->next;
retval = temp_node->val;
current->next = temp_node->next; // リンクの繋ぎ換え
free(temp_node);                // メモリ解放

return retval;
}
```

for ループが $n-1$ 回実行される点に注目してください。これは、 n 番目のノードを削除するために、その一つ手前 ($n-1$ 番目) のノードの `next` ポインタを書き換える必要があるためです。アルゴリズムの目的は、 n 番目のノードではなく、その親となるノードを見つけることなのです。

4. まとめと次のステップ

このトレーニングを通じて、C言語における連結リストの基本を学んできました。最後に、本日の学習内容を振り返り、今後のスキルアップに向けた道筋を示します。

重要点の要約

- 連結リストの概念: 連結リストは、ノードがポインタによって連結された動的なデータ構造であり、配列と比較して要素の挿入・削除に強いという利点があります。
- 基本構造: リストは「データ (val)」と「次へのポインタ (next)」を持つノードから構成され、ヘッドポインタ (head) がリスト全体への入り口となります。
- 基本操作: 以下の基本的な操作のアルゴリズムと、エッジケースを考慮した堅牢なC言語による実装方法を学びました。
 - 探索: リストの先頭から末尾までノードを順に辿る。
 - 追加: リストの先頭または末尾に新しいノードを挿入する。
 - 削除: リストの先頭、末尾、または指定した条件（インデックスや値）に合致するノードを削除する。
 - メモリ管理: 使用後のリスト全体のメモリを正しく解放する。

応用例

連結リストは、より高度なデータ構造を実装するための基礎となります。特に、以下のようなデータ構造は連結リストの応用例として最適です。

- スタック (Stack): 後入れ先出し (LIFO) のデータ構造。リストの先頭への追加 (push) と先頭からの削除 (pop) のみを行うことで効率的に実装できます。
- キュー (Queue): 先入れ先出し (FIFO) のデータ構造。リストの末尾への追加と、先頭からの削除を組み合わせることで実装できます。